

G07 · Nested Learning — 전부 다른 속도의메모리다

이 권은 이 스터디 전체의 “합류점”이다. 앞의 권들은 sequence layer 하나를 어떻게 바꿀지를 다뤘다면, 여기서는 시야를 한 단계 넓혀 **모델과 그모델을 훈련시키는 optimizer를 따로 보지 말고 하나의 중첩 (nested) 시스템으로 보는** 관점을 배운다. 놀라운 재해석 몇 개 (backprop도 메모리다, Adam도 메모리다, attention도 메모리다)를 아주 쉽게 납득하고, 거기서실제로 튀어나온 물건들 — 더 좋은 optimizer, CMS, self-modifying Titans, 그리고 그 합인 **Hope** — 을 시스템 모델링에 바로 쓸 수 있는 형태로정리하는 것이 목표다.

원논문: Nested Learning: The Illusion of Deep Learning Architecture (Behrouz et al., Google Research, arXiv:2512.24695, NeurIPS 2025). 이하 [NL].

1. 한 문장으로: 전부 다른 속도의메모리다

이 권의 큰 통찰은 딱 한 문장이다.

핵심 딥러닝 모델의 모든 부품 — attention, MLP, momentum, Adam, backprop까지—은 사실 “자기 것을 자기 속도로 기억하는 작은 메모리”다. 서로 달라 보이는 각각 메모리가 (1) 무엇을 기억하는지, (2) 어떤 objective로 기억하는지, (3) 얼마나 자주 갱신되는지 — 이 셋만 다르기때문이다. 아키텍처의 다양성은 착시 (illusion)다.

지금까지 우리는 두 가지를 완전히 다른 종류로 취급해 왔다.

- **모델 (아키텍처)**: attention, MLP, SSM… 우리가 “설계” 하는 대상.
- **optimizer(훈련 절차)**: SGD, momentum, Adam, Muon… 모델을학습시키는 “배경 도구”.

[NL]은 이 둘 사이의 벽이 가짜라고 말한다. optimizer도 결국 “과거 gradient를 압축해 저장했다가 꺼내 쓰는 메모리”이고, 모델 layer도 “입력을압축해 저장하는 메모리”다. 둘 다 같은 종류의 물건이고, 단지**위아래 level이 다를 뿐**이다.

비유 회사를 생각하자. 현장 직원은 매 순간 들어오는 주문을 처리한다 (빠른메모리 = attention). 팀장은 하루 단위로 흐름을 요약해 기억한다 (중간메모리). 임원은 분기마다 전략을 갱신한다 (느린 메모리 = MLP weights). 이들은 다른 부서가 아니라 **같은 조직을 다른 시간 스케일로 보는충위**다. Nested Learning은 모델을 이렇게 “속도가 다른 메모리들의중첩”으로 본다.

왜 이게 중요한가? 세 가지 결과가 따라온다. 이 권의 나머지는 이 셋을풀어낸다.

1. 더 좋은 **optimizer**를 “메모리 설계 문법”으로 만들어낼 수 있다 (§ 8).
2. **CMS** — 장기/단기 기억의 이분법을 없애고 “연속적인 주파수스펙트럼”으로 갱신되는 메모리 사술 (§ 9).
3. **self-modifying Titans** — 자기 학습 손잡이 (learning rate, forgetting gate)까지 스스로 바꾸는 메모리 (§ 10).

이 셋을 한 블록에 이어붙인 것이 **Hope**다 (§ 11).

2. 왜 이 이야기가 필요한가: 정적인 LLM

문제의 출발점은 오늘날 LLM 이 배포 후엔 사실상 **아무것도 새로배우지 못한다**는 것이다. 배포된 LLM 에서 지식이 사는 곳은 딱 두 군데다.

- 지금 **context window** — attention 의 KV cache. window 가 밀려나면 사라진다.
- 동결된 **MLP weights** — pre-training 이 끝나는 순간 얼어붙어다시는 안 바뀐다.

비유 [NL] 은 이걸 anterograde amnesia(전향성 기억상실증) 에 비유한다. 방금나눈 대화 (context) 는 잠깐 붙들고 있지만, 그게 장기 저장소 (MLP) 로 넘어가지 못한다. window 가 넘어가면 방금 일은 통째로 잊는다.

뇌는 다르다. 뇌는 **여러 시간 스케일로** 기억을 조직한다: 초 단위, 분 단위, 시간 단위, 그리고 수면 중 재조직까지. 그런데 현대 모델의 “갱신빈도” 는 극단 두 개밖에 없다.

- attention: **매 token 재계산** → 갱신 빈도 = ∞ (제일빠름).
- MLP: **test time 에 동결** → 갱신 빈도 = 0 (아예 안 바뀜).

직관 문제는 “그 사이가 비어 있다” 는 것이다. 매 token 갱신되는 것과 절대 안바뀌는 것 사이에, 100 token 마다 · 1000 token 마다 천천히 갱신되는 중간속도의 메모리가 있어야 하는데 현대 스택엔 그 스펙트럼이 통째로 없다. Nested Learning 은 이 빈 스펙트럼을 채우는 이야기다.

3. “메모리” 를 아주 넓게 다시정의한다

[NL] 의 첫걸음은 “메모리” 라는 단어를 아주 넓게 다시 정의하는 것이다.

직관 메모리 = 입력을 받으면 자기 상태를 조금 바꿔서, 나중에 비슷한 걸 물으면 관련된 걸 되돌려주는 작은 장치. 여기서 핵심은 “**메모리 = 입력이 일으킨 상태 갱신**” 이고, “**학습 = 쓸모 있는 메모리를 얻어가는 과정**” 이라는 것이다. 그러니까 어떤 종류든 gradient 로 상태를 조금씩 바꾸는 일은 전부 “메모리 쓰기” 다.

조금 더 형식적으로, associative memory 는 key 를 주면 value 를 돌려주도록 상태 $\mathcal{M}$ 을 맞춰 놓은 장치다. “얼마나 잘 맞췄나” 를 재는 objective $\tilde{\ell}$ 을 최소로 만드는 상태를 고른다.

$$\mathcal{M}^* = \arg \min_{\mathcal{M}} \tilde{\ell}(\mathcal{M}(K); V)$$

기호 풀이 $\mathcal{M}$ = 메모리의 상태 (예: weight 행렬). K = key 들 (무엇으로 찾을지, 예: 입력 token). V = value 들 (무엇을 되돌려줄지). $\mathcal{M}(K)$ = 그 key 로 메모리를 읽은 결과.

$\tilde{\ell}$ = “읽은 값이 원하는 value 와 얼마나 다른가” 를 재는 손실. $\mathcal{M}^*$ = 그 손실을 최소로 만드는 최적 상태.

이 식은 “key 로 물었을 때 원하는 value 가 최대한 잘 나오도록 메모리 상태를맞춘다” 는 뜻이다. 증명할 것도 없이 그냥 정의다.

여기서 [NL] 이 더하는 결정적인 한 수: **key 와 value 가 꼭 token 일 필요는 없다.** gradient 일 수도, 상위 layer 가 내려보낸 신호일 수도있다. 한 부품이 “압축해서 기억하는 데이터 스트림” 을 그 부품의 **context flow** 라고 부른다.

- sequence layer 의 context flow = token
- optimizer 의 context flow = **gradient**

이 한 줄이 다음 두 절의 “놀라운 재해석” 을 연다.

4. 놀라운 재해석 ①: backprop 도메모리다

이제 “학습 (backprop) 도 메모리” 라는 걸 납득해 보자. 아주 쉽게 간다.

단순한 layer $y = \theta x$ 를 생각하자. 이걸 gradient 로 한 스텝훈련하면 weight 가 이렇게 바뀐다.

$$\theta_{t+1} = \theta_t - \eta_{t+1} u_{t+1} x_{t+1}^\top$$

기호 풀이 θ_t = 시점 t 의 weight. x_{t+1} = 이번 입력. η_{t+1} = learning rate(한 번에 얼마나 크게 고칠지). $u_{t+1} = \nabla_y \mathcal{L}$ = 출력에서의 예측오차 (얼마나, 어느 방향으로 틀렸나 — 이걸 Local Surprise Signal, LSS 라부른다). $u x^\top$ = 두 벡터의 outer product(바깥 곱), 즉 rank-1 행렬하나.

직관 gradient 한 스텝이 결국 “이번 입력 x ” 와 “이번에 틀린 정도 u ” 를 곱한 rank-1 행렬 하나를 weight 에 더하는 일이라는 게 핵심이다. 이걸 정확히 §3 에서 본 “key $x \rightarrow$ value u 를 저장하는 메모리쓰기” 와 같은 모양이다.

이 식은 이런 뜻. backprop 으로 훈련되는 layer 는 사실 “각입력을 봤을 때 얼마나 틀렸는지 (그 예측 오차)” 를 weight 에 하나씩 눌러담는 메모리다. 학습이 란 것도 결국 과거 정보 (무엇을 보고 얼마나틀렸나) 를 압축해 저장하는 일이다.

surprise 라는 단어를 여기서 잡고 가자. u_t = “얼마나 틀렸나” = surprise 의 출력 공간 버전이다. 많이 틀린 (놀란) 입력일수록 weight 를 크게바꾼다 — 놀란 만큼 배운다.

왜 backprop 은 병렬화가 어려운가 (self-reference)

여기서 미묘하지만 시스템 관점에서 아주 중요한 지점이 하나 있다.

linear attention 같은 메모리는 key 와 value 가 **미리 정해져 있어서** 전체를 한 번에 (병렬로) 계산할 수 있다. 그런데 backprop 의 value(예측 오차 u_t) 는 **메모리 자신의 현재 상태가 만들어낸다** — 지금 weight 가 얼마나 틀리는지가 곧 value 니까.

$$v_t = -u_t = -\nabla_{y_t} \mathcal{L}(W_t; x_t)$$

기호 풀이 v_t = 이 스텝에서 메모리에 저장할 target(value). W_t = 지금메모리 상태. 오른쪽은 “지금 상태로 예측했을 때의 오차” 다. 즉 **target** 을 만들려면 **현재 상태** 를 먼저 알아야 한다.

직관 이걸 self-referential(자기참조적) 메모리라고 한다. “다음에 뭘저장할지” 가 “지금 상태” 에 달려 있으니, 스텝 $1 \rightarrow 2 \rightarrow 3$ 을 건너뛰고 한꺼번에 계산할 수 없다. 그래서 backprop 은 순차적이고 병렬화가 어렵다. 이 사실은뒤에서 Hope 의 훈련 트릭 (§ 11 의 chunk snapshot) 을 이해하는 열쇠가 된다.

시스템 모델링 관점 backprop 으로 훈련되는 layer 하나를 모델링하면: **state** = weight 행렬 θ (d^2 규모의 숫자). **write(토큰당)** = rank-1 outer product $u x^T$ 한 번을 더하기 $\rightarrow O(d^2)$ FLOP, KV cache 처럼 길이에 비례해 자라지 않고 크기가 상수. **데이터 의존성** = value 가 현재 state 에서 나오므로 recurrence 가 write 경로 안에 있음 (그래서 순차적). 이건 KV cache(read-only, 길이 L 에 비례해성장) 와 정반대 성질의 state 다 — mutable 하고 크기는 상수.

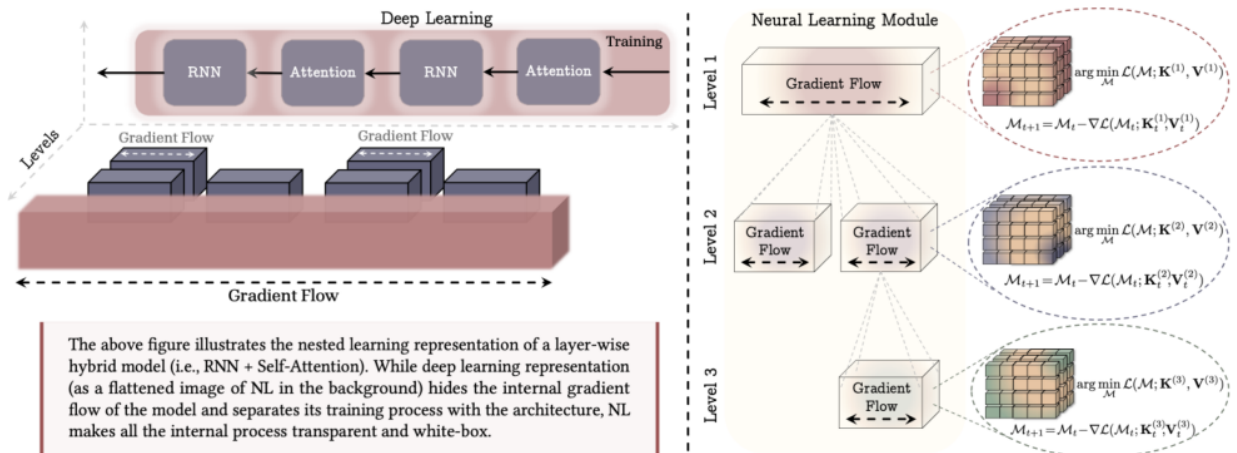
5. update frequency 와 level — 속도로 질서를준다

모델을 “메모리들의 모음” 으로 쪼갬다. 이제 이들에게 질서를 줄 하나의 자가필요하다. 그게 **update frequency**(갱신 빈도) 다.

직관 갱신 빈도 f = “데이터 하나가 들어올 때 이 부품이 몇 번 갱신되나”. attention 은 매 token 새로 계산되니 $f = \infty$. 동결 MLP 는 안 바뀌니 $f = 0$. 그 사이 값을 갖는 부품 (예: 100 token 마다 한 번) 도 있을 수 있다.

빈도로 부품들을 줄 세운다. **빠른 (높은 빈도) 것이 더 낮은 level, 느린 (낮은 빈도) 것이 더 높은 level**. 같은 빈도라도, A 를 계산하려면 B 의 상태가 필요하면 A 가 더 빠른 쪽이다.

이렇게 줄 세운 전체를 **Nested System**, 그리고 그 부품들이 전부 associative memory 인 특수형을 **NSAM (Nested System of Associative Memories)** 이라 부른다. 아키텍처와 훈련 절차를 함께 하나의 NSAM 으로 표현한 설계 단위를 **Neural Learning Module** 이라 한다.



Nested Learning 패러다임. 왼쪽은 hybrid 아키텍처를 평탄화한 deep learning 관점 (내부 gradient flow 가 가려짐), 오른쪽은 같은 모델을 서로 다른 level 의 메모리가 겹쳐 흐르는 투명한 표현으로 펼친 Nested Learning 관점. 출처: Behrouz et al., Nested Learning (arXiv:2512.24695) Fig.2 — 원저자 그림

주의 같은 Transformer 라도 SGD 로 훈련한 것과 Adam 으로 훈련한 것은 [NL] 에겐 다른 **module** 이다. optimizer 가 시스템의 일부이기 때문이다. “모델=아키텍처” 라는 습관을 여기서 버려야 한다.

아래 그림이 이 관점을 요약한다. 왼쪽은 hybrid 아키텍처 (RNN+attention) 를 평소처럼 “평탄하게” 그린 것 — 내부에서 무슨 gradient 가 흐르는지 안보인다. 오른쪽은 같은 모델을 [NL] 방식으로 펼친 것 — 서로 다른속도의 메모리 (최적화 문제) 들이 겹쳐 흐르는 투명한 white-box 다.

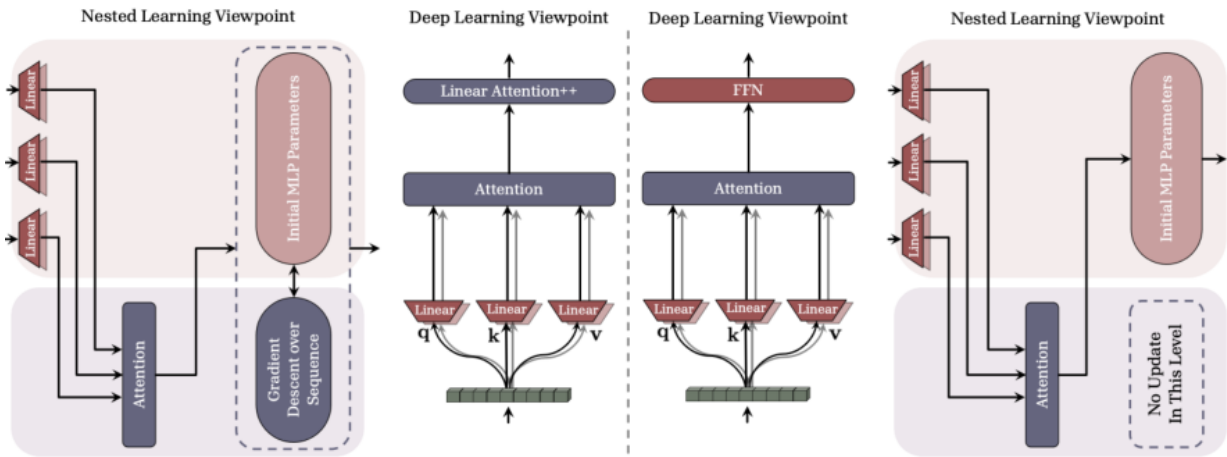
시스템 모델링 관점 이 프레임의 실전 가치: 모델을 “layer 리스트” 가 아니라 “(state, 갱신 빈도, 데이터 의존성)” 의 리스트로 모델링하게해준다. 각 부품에 대해 물어야 할 것 — 이 state 는 몇 바이트인가? 몇 token 마다 갱신되나 (그게 곧 write 트래픽 상황울)? 갱신할 때 어떤 다른 state 를 읽어야 하나 (그게 곧 batching/pipelining 제약)? level 번호는 곧 “얼마나 hot 한 메모리 계층에 들지” 의 힌트가 된다 (§ 13).

6. AdaTransformer: hybrid 아키텍처는착시

“attention + SSM 이중 결합” 이라는 통념도 이 렌즈에선 착시로 풀린다. [NL] 의 예제 하나 (AdaTransformer) 가 이걸 못 박는다.

Transformer block 에서 attention 뒤에 오는 MLP 를 생각하자. 그 MLP 를 “초기상태를 미리 배운 linear attention” 으로 바꿔도 **출력이 대수적으로완전히 똑같다**.

$$y_{\text{block}} = y_{\text{attn}} W_{\text{MLP}} \quad \text{VS} \quad y_{\text{block}} = y_{\text{attn}} W_{\text{LinAttn}}$$



AdaTransformer(Linear Attention++). Transformer 의 FFN 을 초기상태가 meta-learn 된 linear attention 으로 바뀌도 출력이 대수적으로 동일하며, 유일한 차이는 두 weight 가 사는 level (빨강=level 1 context 불변, 파랑=level 2 in-context 갱신) 이다. 출처: Behrouz et al., Nested Learning (arXiv:2512.24695) Fig.3 — 원저자 그림

기호 풀이 y_{attn} = attention 의 출력. W_{MLP} = 보통 MLP 의 weight (동결, context 불변). W_{LinAttn} = linear attention 의 상태 (매 token in-context 로 갱신됨). 두 식의 계산 모양은 같다.

이 식은 이런 뜻. 유일한 차이는 두 weight 가 사는 level 이다. W_{MLP} 은 level 1 ($f = 0$, 안 바뀔). W_{LinAttn} 은 level 2 ($f > 0$, 시퀀스를 따라 갱신됨). 즉 “recurrent 메모리 블록” = “level 이 하나 더 붙은 MLP 블록” 일 뿐이다.

아래 그림이 이 등가를 색으로 못 박는다. 빨강 = level 1(context 불변), 파랑 = level 2(in-context 갱신). FFN 판과 Linear Attention++판은 같은 앞단 attention 위에서 대수적으로 동일하고, 오직 두 번째 weight 가 사는 level 만 다르다.

핵심 hybrid 아키텍처 = “일부 MLP 블록에 in-context 적응 (=level 하나) 이 생긴 Transformer”. 이질적인 부품을 억지로 섞은 게 아니라, 같은 부품에 level 을 하나 더 준 것이다. 이게 제목의 “illusion” 이다.

7. 놀라운 재해석 ②: optimizer 도 전부메모리다

이제 진짜 도발로 간다: momentum, Adam, Muon 같은 **optimizer** 도 전부 “gradient 를 기억하는 메모리” 다. optimizer 의 context flow 가 gradient 라는 § 3 의 한 줄이 여기서 결실을 맺는다.

momentum = 최근 놀란 방향의 관성

momentum GD 를 풀어 쓰면 이렇다.

$$W_{t+1} = W_t + m_{t+1}, \quad m_{t+1} = \beta m_t - \eta_{t+1} g_{t+1}$$

기호 풀이 W_t = weight. m_t = momentum 버퍼 (과거 gradient 를 누적한 것). β = 관성 계수 (과거를 얼마나 남길지, 보통 0.9). η = learning rate. g_{t+1} = 이번 gradient.

직관 momentum = **최근에 놀란 방향의 관성**. 과거 gradient 를 계속 조금씩씩어서 “요즘 대체로 이쪽으로 틀리더라” 를 기억한다. 이걸 정확히 § 3 의 메모리 — gradient 들을 자기 버퍼 m 에 압축하는 associative memory — 다. optimizer state m_t 가 “weight 보다 한 level 아래의 메모리” 인 것이다.

한계 그럼 이 메모리는 얼마나 오래 기억할까? $\beta = 0.9$ 일 때 최근 몇 개가 절반을 차지 하는지 세보면 **약 7개면 50%, 44개면 99%** 다. 즉 momentum 은 장기 기억이 아니라 **최근 44스텝 정도의 단기 기억**이다. 이게 continual learning 실패의 구조적 원인 이다: 새 task 로 옮겨가면 옛 task 방향을 momentum 이 그냥 잇는다. 문제는 모델 용량이 아니라 **optimizer 의 memory management** 다.

주의 실무 함의 하나: “end of pre-training” 에서 optimizer state(m) 를 버리는 관행은, momentum 이 저장해둔 landscape 지식을 통째로 버리는 것이다. continual learning 을 하려면 이 느린 level 도 보존 대상이다.

Adam·Muon 도 같은문법

- **Adam** = 특정 objective (gradient 를 전역 통계로 정규화하는 ℓ_2 regression) 의 **최적 메모리**로 유도된다. 1 차 moment (평균) · 2 차 moment (크기) 를 각각 별도 메모리로 기억하는 것. 단이건 “그 특정 objective 에 대해 최적” 이라는 존재 논증이지, “Adam 이유일한 정답” 이라는 뜻은 아니다.
- **Muon** = “비선형 출력을 단 momentum” . momentum 으로 방향을 모은 뒤 Newton-Schulz 반복으로 직교화하는데, 이 반복 자체가 “momentum 갱신한 번당 여러 스텝을 도는 **내부 최적화 level**” 이다.

핵심 backprop · momentum · Adam · AdaGrad · Muon = 전부 gradient 를 압축하는 메모리. 차이는 (무엇을 기억하나 / 어떤 objective 로 / 얼마나 자주). “학습이란 것도 결국 과거 정보를 압축해 저장하는 일” 이라는 § 1 의 문장이 optimizer 에도 그대로 성립한다.

8. 프레임이 만드는 것 ①: 더 좋은 optimizer

재해석이 옳다면, sequence layer 를 개선하던 손잡이 (objective 바꾸기, delta rule 쓰기, 메모리 깊게 만들기) 를 그대로 optimizer 에 이식할 수 있다. [NL] 은 실제로 몇 개를 만들어낸다.

- **DGD (Delta Gradient Descent)**: 보통 GD 는 각 gradient 를 서로무관하게 취급한다. 하지만 실제 token 들은 서로 상관돼 있어서 낭비가 생긴다. objective 를 dot-product 에서 ℓ_2 regression 으로 바꾸면, GD 에 **data-dependent decay** — “지금 입력과 겹치는 옛 방향을 조금 지우고 쓰기” — 가 자동으로 붙는다. 이게 5 장의 delta rule 을 **learning rule** 층위에서 재발명한 것이다.
- **Delta Momentum**: 같은 아이디어를 momentum 에 적용. momentum 이 자기 상태에 따라 decay 를 조절해, 제한된 용량을 더 잘 관리한다. (linear attention → DeltaNet 전이의 optimizer 버전.)
- **M3 (Multi-scale Momentum Muon)**: momentum 을 하나가 아니라 두 속도로 나눈다. 빠른 momentum(매 스텝) 과 느린 momentum(여러 스텝마다 한 번) 을 각각 유지·직교화한 뒤 합친다.

직관 M3 의 요점: § 7 에서 본 “momentum 은 44 스텝 너머를 못 기억한다” 는 문제를, **decay 를 조절하는 게 아니라 갱신을 아예 늦춰서** 장기 문맥을 얻는식으로 푼다. 이게 바로 다음 절 CMS 를 gradient 에 적용한 것이다 — optimizer 도 “장기/단기 메모리” 를 가질 수 있다는 얘기.

주의 M3 는 품질은 좋지만 (ImageNet-21K ViT 에서 AdamW·Muon 대비 최저 loss) 공짜가 아니다. momentum 이 여럿이고 NS pipeline 이 추가돼서 Muon 보다 느리다. 품질 이득이 계산 비용과 함께 온다.

9. 프레임이 만드는 것 ②: CMS — 연속적인 기억스펙트럼

이제 이 권에서 시스템적으로 가장 중요한 산물. **Continuum Memory System (CMS)** 다.

직관 § 2 에서 본 문제 — 갱신 빈도가 ∞ (attention) 와 0(MLP) 극단뿐이고 사이가 비어 있다 — 를 정면으로 채운다. CMS 는 **여러개의 MLP 블록을 사슬로 잇되, 각 블록이 서로 다른 주기로 갱신되게** 한다. 첫 블록은 자주 (예: 매 chunk), 다음 블록은 더 드물게, 그 다음은 더더욱드물게. “장기 기억 / 단기 기억” 의 이분법을 없애고 **연속적인 주파수 스펙트럼**으로 편다.

각 level ℓ 은 $C^{(\ell)}$ token 마다 한 번씩 갱신된다.

$$C^{(\ell)} := \frac{\max_i f_i}{f_\ell} C$$

(d) Update cadence → memory-tier assignment (★ = recommended)
amortised write energy per token; gated by the FASTER of read/write cadence

fast-weights R+W / token	no cap	★ 11354.82 μJ/tok	19407.88 μJ/tok	24239.72 μJ/tok
CMS-chunk R/tok, W/64	no cap	★ 177.42 μJ/tok	303.25 μJ/tok	378.75 μJ/tok
CMS-mid R+W / 4096	no cap	2.77 μJ/tok	4.74 μJ/tok	★ 5.92 μJ/tok
sleep-experts R+W / 262K	no cap	0.13 μJ/tok	0.22 μJ/tok	★ 0.28 μJ/tok
frozen-weights R/tok, W never	no cap	★ 0.000 μJ/tok	0.000 μJ/tok	0.000 μJ/tok
		SRAM* 60TB/s	HBM3 3.35TB/s	DDR5 0.4TB/s
				CXL 0.5TB/s

■ recommended tier
■ admissible (not cheapest gated)
■ capacity overflow

Source: E1.4 (hatir). *SRAM tier = novel scratchpad twin (simulation_ready=False, directional). Sub-per-token cadence lets a write-heavy RMW state legally drop to DDR/CXL without lengthening the critical path.

갱신 주기 (cadence) 에 따른 메모리 tier 배정. 자주 갱신되는 (짧은주기) 메모리는 on-die/HBM 같은 빠른 tier 에, 드물게 갱신되는 (긴 주기) 메모리는 더 차가운 tier 에 두는 것이 토큰당 상환 write 에너지 기준으로 유리하다 (★=권장). 출처: 본서 저자 자체 실험

기호 풀이 $C^{(\ell)}$ = level ℓ 이 몇 token 마다 갱신되는지 (그 chunk 크기). f_ℓ = level ℓ 의 갱신 빈도. $\max_i f_i$ = 제일 빠른 level 의 빈도. C = 기준 chunk 크기. 즉 느린 level 일 수록 $C^{(\ell)}$ 이 커서 드물게 갱신된다.

이 식은 이런 뜻. 빠른 블록은 몇십 token 마다, 느린 블록은 수천 token 마다 자기 weight 를 조금씩 갱신한다. 중요한 점: 갱신할 때 밟는 gradient step 은 **inference 중에도 밟는 진짜 NTP(다음 token 예측) 손실의 gradient** 다. CMS 는 말 그대로 서빙 중에도 계속 도는 “느린훈련” 이다.

왜 continual learning 을 돕나

비유 도서관을 생각하자. 빠른 블록 = 대출대 앞 신간 매대 (자주 바뀜). 느린 블록 = 서고 깊숙한 아카이브 (거의 안 바뀜). 신간 매대가 새 책으로 덮여도, 옛책은 서고에 남아 있다. 그리고 초기값 backprop 이라는 통로로 서고의 지식이다시 신간 매대로 순환한다. 그래서 완전히 잊는 게 어려워진다. 12 장의 persistent memory 와 “느린 곳에 주차된 지식” 이 하나의 스펙트럼 원리로 합쳐지는 자리다.

CMS 를 어떻게 배선하느냐에 따라 세 변형이 있다 — **Nested**(느린 level 이 빠른 level 의 초기값을 배우고, 문맥이 끝날 때마다 그 초기값으로 reset), **Sequential**(직렬로 잇고 초기값을 한꺼번에 학습), **Independent**(서로 다른 문맥 길이의 블록을 병렬로 두고 가중합으로 결합).

이 스펙트럼을 시스템에 어떻게 매핑하느냐가 곧 메모리 계층 배치 문제다. 아래는 이 스터디의 자체 실험으로, “갱신 주기 (cadence)” 별로 어느 메모리 tier 에 두는 게 에너지상 유리한지를 보여준다.

시스템 모델링 관점 CMS의 갱신은 예정된 시각의 블록에만 일어난다. 그래서 스텝당 건드리는 parameter는 평균 $O(\frac{1}{f} \cdot \frac{L}{5} \cdot d^2)$ 정도 — forward 전체의 작은 조각이다 ($\hat{f}$ =최고 빈도, L =layer 수, d =차원). 그리고 갱신이 안 일어나는 위치들은 서로 순차 의존이 없어 **전부 병렬 처리**된다 (9장 chunkwise 스킴 그대로). 모델링 규칙: “**level 번호 = cache tier 힌트**” — 빠른 level은 hot(HBM 상주), 느린 level(예: 2K token 마다)은 차가운 tier 허용.

참고. 표준 Transformer의 MLP 블록은 CMS의 “블록 1개 + 빈도 0” 인퇴화 사례다. 즉 CMS는 대체물이 아니라 일반화다. 그리고 기존 Transformer의 MLP weight를 CMS 블록의 초기값으로 그냥 꽂는 **ad-hoc level stacking** 배포 경로가 있다 — from-scratch 없이 기존모델을 개조해 쓰는 현실적 이식 스토리다.

10. 프레임이 만드는 것 ③: self-modifying Titans

세 번째 산물. Titans(12장)는 memory를 test-time에 갱신했지만, 그 memory에 들어가는 재료를 만드는 projection(W_K, W_V, W_Q) 과손잡이 (learning rate η , forgetting gate α)는 여전히 동결돼 있었다. self-modifying Titans는 그 마지막 동결을 푼다.

직관 지금까지 학습 손잡이 (η =얼마나 크게 배울지, α =얼마나 잊을지)는 사람이 정하거나 pre-training으로 한 번 정하고 얼렸다. self-modifying Titans는 **이 손잡이들까지 각자 자기 메모리를 갖고, test-time에 문맥을 보며 스스로 바꾼다**. key, value, learning rate, forgetting gate 전부가 “그때그때 학습되는 대상”이 된다.

핵심 한 걸음 (self-modification): 각 memory가 자기 학습 target을 스스로 만든다.

$$\hat{v}_{\square,t} = \mathcal{M}_{\square}(v_t; W_{\square,t-1})$$

기호 풀이 $\square$ =여섯 부품 중하나 ($k, v, q, \eta, \alpha, \text{mem}$). $\mathcal{M}_{\square}$ =그 부품의 memory. $W_{\square,t-1}$ =그 memory의 현재 상태. v_t =입력 재료. $\hat{v}_{\square,t}$ =그 memory가 스스로 만들어낸 학습 target.

이 식은 이런 뜻. 모델이 자기 훈련 데이터를 스스로 만들어, 자기 update 알고리즘을 학습한다. Schmidhuber의 self-referential weight matrix(1993) 계보를 잇는 아이디어인데, 결정적으로 **chunk 병렬화가 가능한 형태로** 만든 게 [NL]의 공학이다.

한계 정직한 각주 하나: query projection q 까지 적응형으로 만들면 오히려 성능이 나빠진다 (ablation에서 ppl 12.24 $\rightarrow$ 12.19로, 동결이 더 좋음). “level을 더 붙이면 항

상 이득”이 아니라는, 논문 자신의 프레임에 대한 내부 반례다. level 배치는 아직 경험적이다.

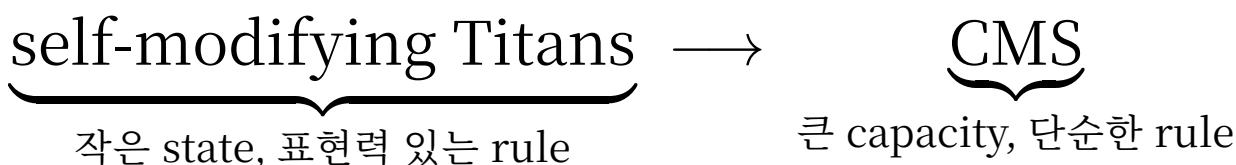
서빙에서의 batchability (쉽게)

시스템 모델링 관점 이게 서빙에 던지는 문제가 크다. self-modifying Titans에서는 **request마다 memory 상태 (weights)가 달라진다**. 보통 LLM 서빙은 “모든 request가 같은 weight를 공유”한다는 전제로 batch를 묶어 큰 GEMM 하나로 돌린다. 그런데 여기서 그 전제가 깨진다.

request들을 batch로 묶을 수는 있다 — 다만 shared-weight matmul이 아니라 **request별로 다른 weight를 쓰는 grouped/batched GEMM**이 된다. 문제는 연산량이 아니라 **대역폭**이다: request마다 다른 weight 텐서를 HBM에서 읽어와야 하니, 묶어봐도 weight read 트래픽이 request 수에 비례해 그대로 늘어난다. 그래서 **batching으로 얻는 이득이 제한적**이다. (KV cache는 request별로 다르지만 read-only라 이 문제가 덜하다. 여기 state는 mutable weights라 성질이 다르다.)

11. Hope = 셋의 결합

이제 셋을 합친다. Hope는 한 블록 안에 두 산물을 직렬로 잇는다.



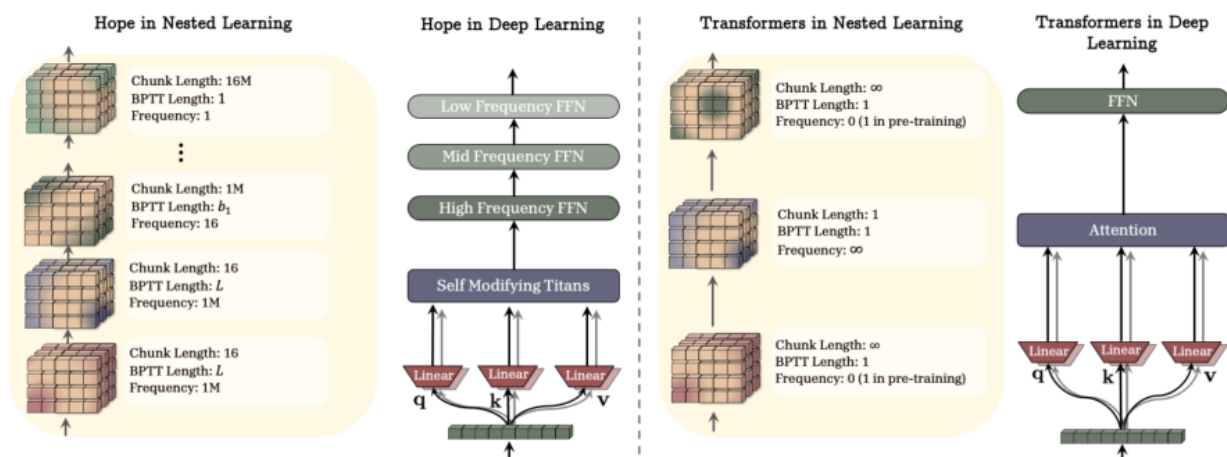
직관 둘은 상보적이다. self-modifying Titans는 state가 작아서 표현력 있는 (DGD) rule을 감당할 수 있고, CMS는 지속 저장을 위해 capacity가 커서 단순한 rule을 쓴다. 앞에서 문맥을 똑똑하게 압축하고, 뒤에서 주파수 스펙트럼으로 오래 보존한다.

아래 그림이 Hope의 backbone을 Transformer와 나란히 놓는다. self-modifying Titans 뒤에 high/mid/low frequency FFN 사슬 (CMS) 이어지고, 각 블록에 chunk length·frequency가 붙어 있다 — Transformer의 두 극단 ($f = \infty$ attention, $f = 0$ FFN)을 하나의 연속 스펙트럼으로 대체한 것이다.

훈련은 어떻게 병렬화하나 (chunk snapshot)

§ 4에서 “self-referential 메모리는 순차적이라 병렬화가 어렵다”고 했다. 그럼 self-modifying Titans는 어떻게 훈련이 병렬화되나?

직관 답은 타협이다. 시퀀스를 chunk로 자르고, 한 chunk 안에서는 “target을 만드는 상태”를 직전 **chunk 끝의 snapshot**으로 올려둔다. 그러면 그 chunk 전체의



Hope의 아키텍처 backbone과 Transformer 비교. Hope는 self-modifying Titans(작은 state, 표현력 있는 rule) 뒤에 서로 다른 update frequency의 FFN 사슬 (CMS: high/mid/low frequency)을 직렬로 잇는다. long/short-term 이분법이 주파수 연속체로 펼쳐진다. 출처: Behrouz et al., Nested Learning (arXiv:2512.24695) Fig.5 — 원저자 그림

key·value·target·gate를 한 batch로 미리 계산할 수 있어 병렬화된다. 정확한 self-reference(매 순간 현재상태로 target 생성)는 여전히 순차적이지만, chunk 경계 snapshot 근사가 병렬성을 사준다. chunk 크기 C 가 충실도 $\leftrightarrow$ throughput 다이얼이다.

한계 이 근사가 정확한 self-modifying 모델을 얼마나 훼손하는지 (오차 한계)는 논문에 없다. SRWM이 정확한 self-reference를 갖고도 병렬 훈련이 어려웠던 걸, Hope는 이 근사 덕에 키운다 — 대가의 정량은 미측정.

12. 결과: “더 많은 layer”가 아니라 “더 많은 level”

[NL]의 핵심 실증 주장은 이거다: 성능을 사는 건 layer를 더 쌓는 게 아니라 **level**을 더 붙이는 것이다.

아래 그림이 그걸 보여준다. memory level 수를 1 $\rightarrow$ 4로 늘리면 세 long-context 벤치마크에서 성능이 단조 개선되고, 어떤 level 수 · 최저주파수에서도 ICL·DuoAttention baseline을 상회한다.

주요 숫자 몇 개 (시스템 모델링에 감을 주는 용도):

항목	결과
언어 모델링 (1.3B/100B)	Hope가 전 baseline 최고 (Wikitext ppl 14.39, 상식추론 평균 58.0)
RULER 긴 문맥 needle	attention-free 중 전 과제 최고 — deep memory capacity가 버팀
BABILong(fine-tune)	Hope가 10M token까지 유지 (Titans·ARMT는 급락)

항목	결과
continual learning(CTNL)	level 수에 단조 개선; Hope-3가 단일 언어 성능 거의 회복
level 최저 주파수	512가 최고, 2K가 근접 성능에 훨씬 저렴 (효율 sweet spot)

한계 정직하게 짚을 것 셋. (1) 짧은 **in-context recall** 은 여전히 Transformer가 이긴다 (FDA 67.3 vs Hope 41.9). Hope는 attention-free 중최고로 격차를 좁혔을 뿐, in-context retrieval gap은 미해소다. (2) BABILong 10M은 **fine-tuning** 전제 — zero-shot이면 다 떨어진다. (3) **Hope의 decode throughput·latency·메모리 수치가 논문에 없다**. wall-clock 측정은 M3(훈련 시간) 뿐. from-scratch 상한은 1.3B/100B.

13. 시스템 모델링 관점총정리

이 권에서 시스템 엔지니어가 실제로 들고 갈 것들을 한자리에 모은다.

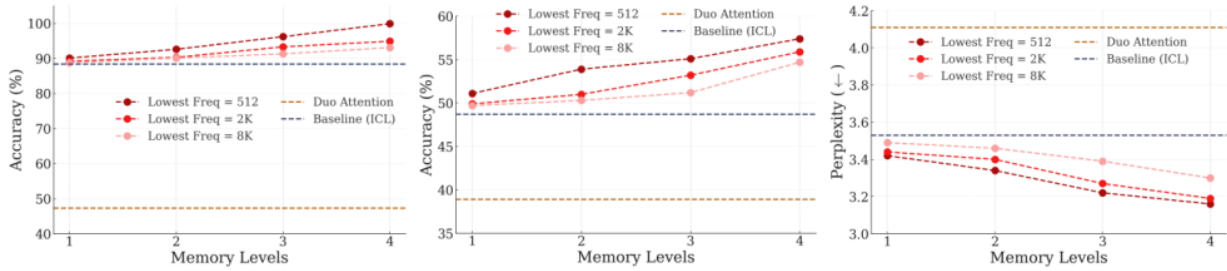
시스템 모델링 관점 (state 산수). Hope 블록 하나의 inference state = 여섯 개의 2-layer residual MLP memory(memory 당 행렬 2개 → 도합 약 12개의 $d \times d_h$ 급 행렬) + inference 중 표류하는 CMS level 별 MLP weights. KV cache 처럼 $O(L \cdot d)$ 로 자라지 않고 **시퀀스 길이에 상수**다 — 다만 그 상수가 weights 규모라 crossover 지점이 단일 matrix memory 보다 훨씬 오른쪽이다. Titans/TTT/Atlas와 같은문제 클래스이되 fast-weight 텐서가 약 6배.

state가 KV cache 나 weights 나에 따라 트래픽 경제가 갈린다. 아래는 자체실험 — memory state S 가 커질수록 KV cache 방식(read-only, 길이에비례) 보다 TTT/weights 방식(상수 state, RMW)이 유리해지는 crossover 를보여준다.

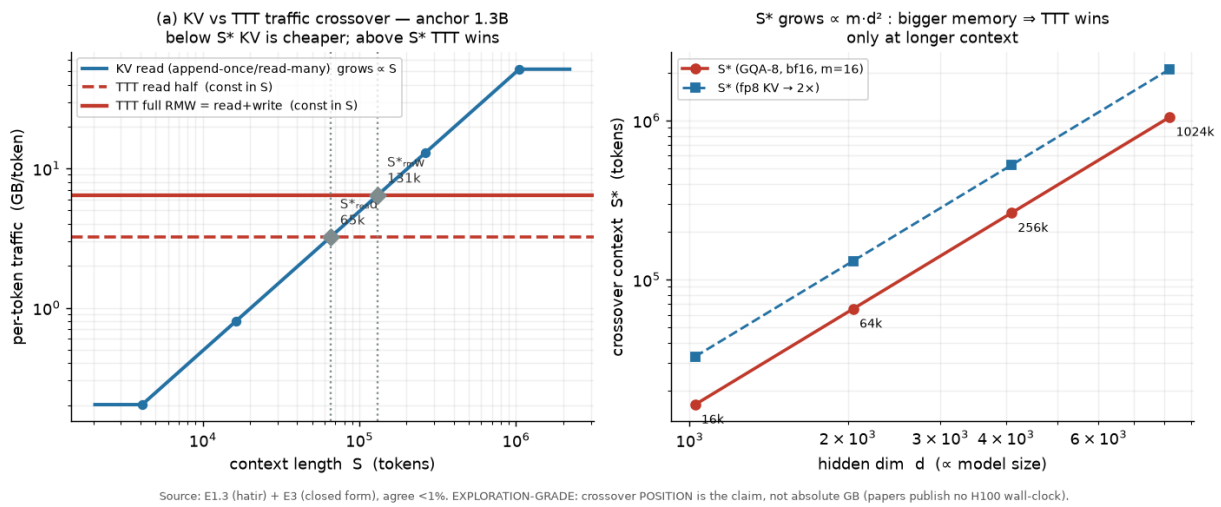
시스템 모델링 관점 (write 트래픽). 각 fast memory는 $C_{\square}$ token 마다 자기 parameter 텐서 전체를 read-modify-write(RMW) 한다. 따라서 **토큰당 상환 write 트래픽 = (state bytes) / $C_{\square}$** . chunk 크기 C 가 곧 대역폭 다이얼이다. “최저 주파수 2K가 sweet spot”이라는 실험 결과는, 이 트래픽을 크게 상환해도 품질을 별로 안 잃는다는 뜻 — 시스템 친화적 신호다.

RMW에는 함정이 하나 있다. state가 on-die(SRAM/캐시)에 들어가느냐 off-die(HBM)로 넘치느냐의 경계에서 대역폭이 절벽처럼 떨어진다. 아래 자체실험이 그 cliff를 보여준다.

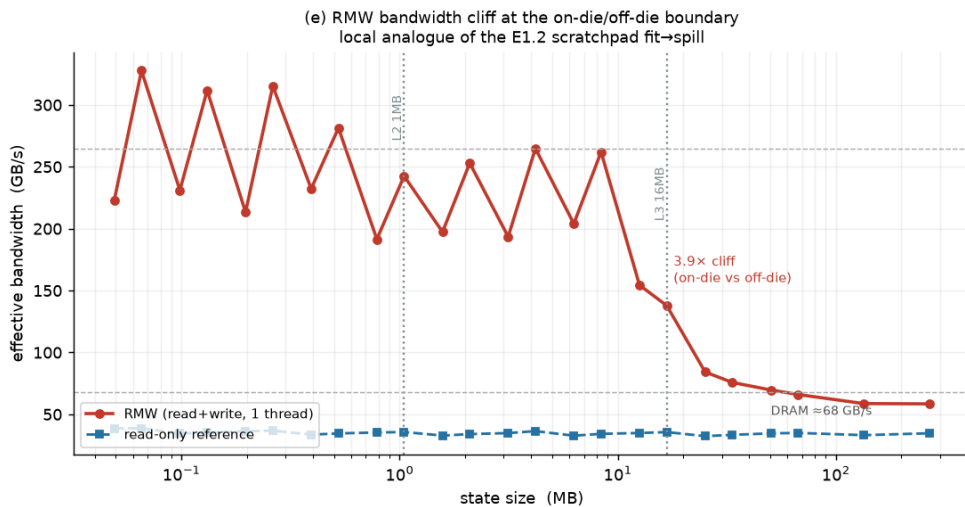
시스템 모델링 관점 (batching과 kernel). (1) **batching** 이깨진다. state가 read-only cache가 아니라 request 별 mutable weights라, shared-weight batching이 성립 안 한다. 묶어도 grouped-GEMM이 되고 weight read 대역폭 이득은 제한적 (§ 10). (2) **kernel 형태**. 훈련·prefill은 chunkwise dual 스킴 — memory가 MLP이므로 TTT-MLP형 batched small-GEMM 시퀀



memory level 수가 in-context learning 성능에 미치는 효과 (좌: RULER MK-NIAH, 중: LongHealth, 우: QASPER — 우측은 perplexity라 낮을수록 좋음). level 이 많을수록, 최저 주파수가 낮을수록 성능이 오르고 baseline(점선)을 상회한다. 출처: Behrouz et al., Nested Learning (arXiv:2512.24695) Fig.7 — 원저자 그림



KV cache vs TTT/weights 트래픽 crossover(anchor 1.3B). memory state가 임계 크기 S^* 아래면 KV cache가 싸고, S^* 위로 커지면 상수-state RMW 방식이 이긴다. 출처: 본서 저자 자체 실험



Source: E2.2 (CPU numpy). SHAPE-only: the 3.9x L3→DRAM cliff structure transports; GB/s values do not (H100 on/off-die ridge ~100x elsewhere). RMW = 0.5x read throughput = the write-back tax KV avoids.

RMW 대역폭 절벽 (on-die/off-die 경계). memory state가 on-die 용량에 맞으면 RMW가 빠르지만, 경계를 넘겨 off-die로 spill 하면 유효대역폭이 급락한다. 출처: 본서 저자 자체 실험

스, linear 특수 사례는 DeltaNet 계열 WY/UT-transform chunk kernel 이 자연 템플릿. (3) **decode 비용은 미측정**. 이상적 $C = 1$ decode 는 매 token 순차 (상환 없음), $C > 1$ decode 는 C 개 버퍼링 후 경계 지연 batch-update — 정확한 latency 는 공개 구현이 없어 미확정.

주의 (가장 파괴적인 함의). continual 배포에서 모델은 **서빙 중 사용자 데이터로 자기 NTP gradient step 을 계속 밟는다** — train/serve 경계의 소거. 이걸 서빙 불변식의 영구 폐기다. per-tenant weight 발산, rollback, provenance, 안전성, speculative decoding 상호작용 — 어느 것도 논문이 다루지 않는다. 이 라인의 서빙 문제는 종합판에서 풀리기는 커녕 6 배로 늘었다.

14. 이 숫자를 믿어도 되나 — 성능 수치의 신뢰성

§ 13 의 두 그림 (crossover, RMW cliff) 은 “본서 저자 자체 실험” 이다. 발표에서 이 수치를 쓰려면 “**믿을 만한가**” 를 두 층위로 쪼개 물어야 한다 — (A) 계산 엔진 (HATIR) 이 맞게 계산하나, (B) 넣은 하드웨어 스펙 (HAT twin) 이 실제 HW 를 맞게 표현하나. 둘은 완전히 다른 질문이고, 답도 다르다.

(A) HATIR 계산은 독립 검증됐다 — 단 “traffic 계수” 까지

HATIR 는 roofline/traffic cost model 이다. 이게 맞는지 **ZigZag** (KU Leuven 의 공개 학술 cost model, pip install zigzag-dse) 라는 독립 **oracle** 과 같은 HW·workload·mapping 에서 맞대 봐 검증했다. 결과: **operand 별 DRAM traffic 바이트를 0.00% 오차 (바이트 단위 정확) 로 재현**, MAC 수 정확, memory-bound/compute-bound 분류 일치 (GEMM 3 형태 + spill 케이스). 남은 gap 은 corner-tile 나머지뿐 ($\leq 15\%$).

핵심 검증된 것은 **traffic 계수 (몇 바이트가 오가나) 와 bound 분류**다 — § 13 crossover·cliff 가 딛고 선 바로 그 양. 검증 안 된 것은 절대에너지 (pJ, 기술 노드 의존) 다. 즉 “얼마나 오가나” 는 독립 확인됐고, “그게 몇 줄 (J) 이냐” 는 아니다.

(B) HAT spec 은 NVIDIA 공식 문서 출처 — 단 datasheet peak 값

twin dgx_h100_x4.json 의 숫자는 **NVIDIA H100/GH100 whitepaper + DGX 시스템 가이드 + Hopper 튜닝 가이드 + CUTLASS** 에서왔다 (HBM3 3.35 TB/s, 7 pJ/B, L2 50 MB, ridge 295 FLOP/B 등). 정직하게 말하면 이걸 **datasheet 규격 (peak-nominal)** 이지, 특정 실물 한 대를 측정한 값이 아니다. 그리고 아직 필드별 provenance 태그가 붙은 건 일부뿐이다.

가장 중요한 gap — ideal(스펙) vs 실측 (achieved)

raw roofline 은 **이용률 100%**를 가정한다. 실측과 대면 결과:

양	HATIR ideal	실측 H100	낙관 정도
GEMM (compute-bound)	959 TFLOPS	716 (cuBLAS, 72.4%)	~34% 과대
batch-1 decode MBU (memory-bound)	(peak)	26.9%	더 큼

즉 raw HATIR 절대치는 실제보다 대략 1.3 – 1.4 배 낙관적인 “이상적하한” 이다. 중요한 건 이 gap 을 다루는 방식이다 — 이 프로젝트는 MFU/MBU 같은 **fit 상수**를 거부한다. (“H100+CUDA 의 특성을 상수에 인코딩해없는 칩으로 몰래 수입하는 것 = 방법론적으로 무효.”) 대신 gap 을 출처 (provenance) 별로 분해한다: computed(설계식에서 결정론적 계산: wave/tile quantization·occupancy), technology(HBM 컨트롤러 이용률 ~0.7 – 0.9 등 기술 전이값), stack(kernel-launch floor·scheduler·CUDA-Graphs 같은 SW 오버헤드), residual(남는 것 — 크기를 보고하고 절대 소급 **fit 안 함**). MFU 0.75 를 넣으면 719 TFLOPS 로 실측과 0.46%까지 불지만, 그상수는 default-off 이고 core 경로는 ZigZag byte-exact 를 유지한다.

그래서 발표에서 무엇을 단정하고 무엇을유보하나

이게 결론이자 발표 규칙이다. 모델 오차가 상쇄되는 양만 단정하고, 안 되는 양은 유보한다.

믿어도 되는 것 (단정 가능). - **crossover S^*** — 이걸 “KV read 바이트 = TTT RMW 바이트” 라는 순수 **traffic** 등식 (GB = GB) 이다. 양변이 같은 대역폭 · 같은 MFU/MBU 를 타므로 **BW 스펙값도, 이용률도 전부 상쇄**된다. twin BW 가 틀려도, 실측 MFU 가 50%여도 S^* 는 그대로다. ZigZag 가 traffic 계수를 byte-exact 로 검증했으니 이 양은 견고하다. - **bound** 분류 — decode AI 0.59 vs ridge 295 는 2 배 스펙 오차로도 안 뒤집힌다. - **tier** 순서 · 비율 — 상대비교라 공통 배수가 상쇄된다.

유보할 것 (절대 성능으로 말하지 말 것). - **μ s/token, mJ/token, GB/s** — 전부 ideal 하한이고 실측은 ~1.3 – 1.4 배나 빠르다. datasheet peak 에 기반한 값이라 실물 fidelity 는 별도 검증 (사내 A100 runbook) 으로만 메운다. 발표에선 “이상적 하한, 실측 예정” 으로만 인용.

한 줄로. § 13 의 crossover·cliff 는 구조 (어디서 뒤집히나, 무엇이 memory-bound 나) 를 말하는 그림이지 절대 속도를 재는 그림이 아니다. 이 라인에서 우리가 단정하는 건 상쇄로 견고한 구조뿐이고, 절대성능은 정직하게 열어 둔다. (근거 · 재현: 본서 부록 E, experiments/REPRODUCE.md.)

15. 한계와 다음권으로

논문이 스스로 그은 한계선을 정리한다.

한계 (1) catastrophic forgetting 은 해결되지 않았다 — CMS·Hope 는 forgetting 을 “줄였을” 뿐, 어디서 · 얼마나 빨리 잊는지를 주파수 축에재배치한 것이지 없앤 게 아니다. (2) level 설계가 경험적이다 — level 수 · 주파수 · 배치에 이론이 없고 inner q 반례도 있다. (3) 이론은 재해석까지 — optimizer 역공학은 존재 논증이고, level 수에 따른 표현력 정리는 없다 (formal language 100%는 실증이지 정리가아님). (4) 서버 경제학 · 스케일 미검증 — wall-clock 부재, mutable-weights 인프라 미해결, 1.3B/100B 상한.

요약 Nested Learning 은 “모델과 optimizer 를 하나의 중첩 시스템으로, 각 부품을 자기 속도로 자기 것을 기억하는 메모리로” 보는 관점이다. 이 렌즈에서 backprop·momentum·Adam·Muon 은 전부 gradient 를 기억하는 메모리이고, attention·linear attention·Titans 는 한 단계 위 메모리이며, 차이는 오직 (무엇을 / 어떤 objective 로 / 얼마나 자주) 셋뿐이다. 관점 자체는 검증불가한 선언에 가깝지만, 그 관점이 **생성한 물건** — DGD/M3(더좋은 optimizer), CMS(장기/단기를 연속 스펙트럼으로), self-modifying Titans(자기 손잡이도 스스로 바꾸는 메모리), 그리고 그 합 Hope — 은 ablation 으로 기여가 측정된 구체물이다. 시스템 관점의 알맹이: state = request 별 mutable weights(길이 상수, 크기 = weights 규모), write 트래픽 = state bytes / chunk 크기, level 번호 = memory tier 힌트, 그리고 batching·serving 불변식이 근본부터 깨진다는 것.

다음 권 예고 — G08 · Sleep. [NL] 은 “입력이 흐르는 동안의 online consolidation” 만 구현하고 수면 중 재조직 (offline consolidation) 은범위 밖으로 선언했다 — 그래서 “빠른 level 의 지식이 덮어써지기 전에어디로 옮기나?” 라는 질문을 열어둔 채 남긴다. 다음 권은 그 답, 즉 wake/sleep lifecycle 과 잊히기 전에 지식을 옮기는 절차를 다룬다.